

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I P. Guru/192521010 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Guru/

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming

computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. **A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.**

ANS:

1. Big Data Platform for Website Clicks, Mobile Interactions and Transactions

- A big data platform that collects billions of website clicks, mobile interactions, and customer transactions every day would require a highly distributed architecture capable of handling large volume, velocity, and variety of data. By reverse engineering such a system, the probable architecture would consist of distributed storage, high-speed data ingestion, batch and stream processing, analytics engines, and business intelligence tools.
- The distributed storage system is most likely based on Hadoop Distributed File System (HDFS) or a modern cloud-based data lake such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. Raw clickstream logs, mobile events, JSON files, transaction records, and application logs can be stored in their original form in the data lake. HDFS divides large files into blocks and distributes them across multiple nodes, providing fault tolerance through replication. In a cloud environment, object storage provides similar scalability and durability.
- For data ingestion, the platform would probably use Apache Kafka or similar messaging systems. Website clicks, mobile application events, and transaction events can be continuously published as messages to Kafka topics. Different topics could be created for clicks, payments, customer activity, and transactions. Kafka partitions each topic, allowing multiple consumers to process data simultaneously. This makes it suitable for billions of events arriving continuously.
- For batch processing, Apache Spark would probably be the primary processing framework. Spark can process very large datasets in parallel across a cluster. Historical customer transactions could be analyzed to identify purchasing patterns, calculate sales performance, and generate customer segments. Hadoop MapReduce could also be used for traditional batch workloads, although Spark is generally preferred for faster iterative analytics.
- The platform would separate structured and semi-structured data according to their characteristics. Structured transaction data, such as customer ID, product ID, quantity, price, and transaction date, could be stored in relational databases or analytical warehouses such as PostgreSQL, Snowflake, BigQuery, or Amazon Redshift. Semi-structured information such as JSON clickstream events and mobile logs could be stored in a data

lake using formats such as JSON, Avro, Parquet, or ORC. Parquet is especially useful because it supports column-based storage and efficient analytical queries.

- Indexing and partitioning would improve query performance. Transaction data could be partitioned by date, region, or customer segment, while clickstream data could be partitioned by event date and application. Frequently queried attributes could be indexed or organized using metadata catalogs. Distributed query engines such as Presto/Trino or Spark SQL could then execute analytical queries across large datasets without moving all the data to one system.
- Real-time analytics would require stream-processing engines such as Apache Flink, Spark Structured Streaming, or Kafka Streams. These engines could calculate real-time website traffic, active users, transaction rates, fraud indicators, and customer behavior. For example, if a sudden increase in transactions occurs from a particular region, the streaming system can detect it immediately and update an operational dashboard.
- Scalability would mainly be achieved through horizontal scaling. Instead of depending on one powerful server, the platform would add more nodes when data volume increases. Kafka partitions, Spark executors, and distributed storage nodes allow workloads to be divided across multiple machines. Replication provides fault tolerance, while load balancing distributes requests among available servers.
- Finally, dashboards and business intelligence systems such as Power BI, Tableau, Looker, or Grafana would connect to the analytical warehouse or serving layer. Instead of directly querying raw data, dashboards would usually access processed datasets, aggregated tables, or OLAP models. Business managers could therefore monitor KPIs such as sales, conversion rate, customer acquisition, website traffic, and revenue in near real time. This architecture provides a complete flow from data collection → ingestion → storage → processing → analytics → visualization, allowing the organization to make faster and more informed decisions.

2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline

ANS:

E-Commerce Platform for Cart, Checkout, Product View and Payment Analytics

- An e-commerce platform handling product views, cart additions, abandoned checkouts, and payment behavior requires a big data architecture capable of processing both real-time and historical customer information. Reverse engineering such a system suggests the use of event streaming, distributed storage, customer databases, caching systems, machine learning, and analytical dashboards.
- The primary event-streaming tool would likely be Apache Kafka. Every customer action can be represented as an event. For example, viewing a product generates a `product_view` event, adding an item creates a `cart_addition` event, and completing payment produces a `payment_success` event. These events can be sent to separate Kafka topics. Kafka partitions the topics so that multiple processing nodes can consume events simultaneously.
- Customer profiling would probably use a combination of relational databases, NoSQL databases, and analytical stores. Structured information such as customer ID, address,

order amount, payment status, and product quantity could be maintained in databases such as MySQL, PostgreSQL, or cloud data warehouses. A NoSQL database such as MongoDB, Cassandra, DynamoDB, or Redis could maintain rapidly changing customer profiles, preferences, sessions, and personalization information.

- Session data, clickstream logs, and transactions need to be synchronized for real-time personalization. A customer viewing several mobile phones, adding one to the cart, and abandoning checkout generates a sequence of related events. Kafka can transport these events while Flink, Spark Structured Streaming, or Kafka Streams can process them in real time. A customer profile store can then be updated with the latest behavior. This enables the platform to immediately display related products, discounts, reminders, or recommendations.
- Caching is extremely important in e-commerce because frequently accessed information should not always be retrieved from the main database. Redis is a likely caching technology. Product information, shopping carts, user sessions, recommendation results, and frequently viewed products can be stored in Redis. This reduces database load and improves response time.
- For distributed analytical queries, systems such as Trino, Presto, Spark SQL, BigQuery, Snowflake, or Amazon Athena could be used. Historical browsing and transaction data can be stored in a data lake using Parquet or ORC files. The query engine can then process large datasets efficiently.
- The recommendation workflow would normally involve several machine learning stages. First, raw clickstream and transaction data are cleaned and transformed. Next, customer features such as purchase frequency, preferred categories, average order value, browsing history, and abandoned carts are generated. Recommendation algorithms such as collaborative filtering, content-based filtering, association-rule mining, or neural recommendation models can then predict products likely to interest each customer.
- Structured order data and semi-structured browsing data can be merged using common identifiers such as customer ID, product ID, session ID, and timestamp. An ETL or ELT pipeline can combine transaction tables with JSON-based clickstream events. This produces a unified customer behavior dataset for analytics and machine learning.
- Scalability is achieved through partitioning, replication, distributed processing, and elastic cloud resources. Kafka partitions distribute events, databases can use sharding, and Spark or Flink distributes processing across multiple workers. Fault tolerance can be achieved through Kafka replication, database replicas, checkpointing, and retry mechanisms. If one processing node fails, another node can continue processing the workload.
- Finally, dashboards can be generated using Power BI, Tableau, Looker, Grafana, or similar BI platforms. KPI dashboards may show conversion rate, cart abandonment rate, average order value, revenue by region, product performance, customer retention, and payment success rate. Real-time dashboards can consume aggregated streaming results, while

historical dashboards can query the data warehouse. Thus, the architecture supports both immediate personalization and long-term business analysis.

3.A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance

requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

ANS:

Big Data Healthcare System for Patient Histories, Images, Sensors and Prescriptions

- A healthcare big data platform must manage different types of sensitive information, including patient histories, diagnostic images, wearable sensor readings, laboratory results, and prescription records. Since these datasets have different structures and storage requirements, a hybrid database architecture would most likely be used.
- Structured information such as patient ID, age, diagnosis codes, prescriptions, appointment details, laboratory values, and billing information could be stored in relational databases such as PostgreSQL, MySQL, Oracle, or SQL Server. These databases provide ACID transactions and strong consistency, which are important for critical healthcare records.
- Unstructured information such as X-rays, CT scans, MRI images, scanned documents, and medical reports would be stored in distributed object storage or specialized medical repositories. Diagnostic images may use the DICOM standard, which is specifically designed for medical imaging. A data lake could store large medical files while metadata about those files is maintained in a relational or NoSQL database.
- Healthcare organizations such as hospitals, laboratories, insurance companies, pharmacies, and clinics need interoperability. Standards such as HL7 and FHIR can be used to exchange electronic health information. Integration tools and APIs can connect hospital information systems, laboratory systems, electronic health records, and insurance applications. An ETL/ELT platform can extract information from different systems, transform it into common formats, validate it, and load it into a centralized analytical environment.
- Wearable devices produce continuous streams such as heart rate, oxygen saturation, temperature, activity, and sleep information. These high-velocity streams could be transmitted through Apache Kafka, MQTT, or cloud IoT services. Stream-processing systems such as Apache Flink or Spark Structured Streaming could analyze the information immediately. If a patient's heart rate crosses a dangerous threshold, an alert can be generated for healthcare personnel.
- Machine learning can be applied at different stages. Historical patient records can be processed using Spark to create large training datasets. Machine learning algorithms can then identify risk factors associated with diseases, readmission, complications, or deterioration. More advanced deep learning models can analyze medical images to assist in identifying abnormalities. Predictive models can also estimate disease risk based on laboratory results, patient history, and wearable sensor information.
- Privacy and security are critical because healthcare information is highly sensitive. Data should be encrypted at rest and in transit using strong encryption mechanisms. Access

should follow role-based access control so that doctors, nurses, administrators, laboratories, and insurance staff only access information required for their responsibilities. Multi-factor authentication, secure APIs, network segmentation, and identity management further protect the system.

- Audit logging is also important. Every access, modification, download, or transmission of patient data should be recorded. Data masking, anonymization, and pseudonymization can be applied when information is used for research. Compliance requirements depend on the country and organization, but healthcare platforms commonly need to follow applicable regulations such as HIPAA or local data-protection requirements.
- The analytics pipeline could follow the sequence data sources → integration → secure storage → preprocessing → distributed processing → machine learning → clinical analytics → visualization. Doctors could use dashboards to monitor patient risk, laboratory trends, medication information, and alerts. Population health researchers could analyze large datasets to identify disease patterns, risk factors, and healthcare trends.
- Therefore, the likely architecture combines relational databases for structured records, distributed storage for medical files, streaming technologies for wearable data, distributed processing for large-scale analytics, and strong security controls. Such an architecture can support both individual clinical decision-making and population-level healthcare research.

4.A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

ANS:

Smart City Platform for Traffic, CCTV, Weather and Public Transport

- A smart city platform continuously collects information from traffic sensors, CCTV systems, weather stations, GPS-enabled public transport, parking systems, and other IoT devices. Because the data is generated continuously and often requires immediate analysis, the probable architecture would combine edge computing, distributed messaging, cloud storage, stream processing, geospatial databases, and visualization systems.
- Edge computing would be placed close to sensors and cameras. Instead of sending every raw video frame to a centralized cloud, edge devices could perform initial processing such as vehicle counting, object detection, traffic density estimation, and abnormal-event detection. This reduces bandwidth consumption and lowers latency. Only important metadata or detected events may be transmitted to the central platform.
- A distributed messaging system such as Apache Kafka, MQTT, or Apache Pulsar could collect sensor events. Traffic sensors might generate vehicle-count events, weather stations could produce temperature and rainfall events, and public transport systems could generate bus location events. Kafka topics could separate these streams, while partitions allow multiple processing nodes to consume data concurrently.
- Centralized storage would probably consist of a combination of cloud object storage and databases. Historical sensor data can be stored in a data lake. Time-series databases such as InfluxDB or TimescaleDB are suitable for storing continuously changing measurements. Location-aware information can be stored in PostGIS, MongoDB, or other geospatial databases. These systems allow queries based on geographic coordinates, routes, regions, and distances.
- Streaming frameworks such as Apache Flink or Spark Structured Streaming could process traffic data in real time. For example, the system can calculate average vehicle speed, traffic density, queue length, and congestion levels. By combining current traffic information with weather and historical traffic patterns, machine learning models can predict congestion before it becomes severe.
- Geospatial analytics is particularly important. Traffic events can be associated with road segments, intersections, GPS coordinates, and geographic regions. GIS technologies can visualize congestion on digital maps. Spatial queries can identify the nearest emergency vehicle, congested roads within a specific region, or alternative routes around an accident.
- Batch and real-time analytics would operate together. Real-time analytics could support immediate traffic management, while batch analytics could analyze months or years of historical data. Urban planners could use historical information to identify locations

where new roads, public transport routes, or traffic signals are required. Machine learning models trained on historical data could also improve future congestion predictions.

- Security is essential because smart city infrastructure contains sensitive operational information. Devices should use secure authentication and encrypted communication. TLS encryption, API gateways, firewalls, network segmentation, role-based access control, and identity management can protect the system. CCTV data should receive additional privacy protection, particularly when personally identifiable information may be involved. Audit logs can track access to sensitive systems.
- Visualization systems such as Grafana, Power BI, Tableau, or custom GIS dashboards can display live operational intelligence. Traffic control centers could see current congestion levels, accidents, weather conditions, public transport locations, and sensor failures on a single dashboard. Alerts can automatically notify authorities when congestion exceeds a threshold or infrastructure failure is detected.
- Thus, the architecture combines edge processing for low latency, distributed messaging for reliable ingestion, cloud or distributed storage for massive historical datasets, streaming analytics for immediate decisions, and batch processing for long-term urban planning. This provides city authorities with both real-time operational awareness and strategic insights.

5.A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

ANS:

Healthcare Analytics System for Records, Wearables, Labs and Appointments

- A healthcare analytics system integrating patient records, wearable readings, laboratory reports, and appointment histories requires a hybrid big data architecture because the information is both structured and unstructured. It must also support real-time monitoring while maintaining strict privacy, security, and governance.
- Structured patient information such as patient ID, demographic information, appointment details, laboratory values, diagnosis codes, prescription information, and billing records can be stored in relational databases such as PostgreSQL, MySQL, Oracle, or SQL Server. Relational systems provide strong consistency and transaction management, which are important for healthcare applications.
- Wearable devices generate continuous measurements such as heart rate, blood pressure, oxygen saturation, activity level, and sleep patterns. These data streams can be transmitted through IoT gateways using protocols such as MQTT and then ingested through Apache Kafka or cloud streaming services. Unstructured or semi-structured information such as medical reports, documents, and device-generated JSON records can be stored in a data lake using object storage.
- The system would require ETL or ELT pipelines to integrate data from hospitals, clinics, laboratories, pharmacies, and wearable platforms. Tools such as Apache NiFi, Talend, Informatica, or cloud-based data integration services could extract, transform, validate, and load information. Healthcare interoperability standards such as HL7 and FHIR can provide consistent communication between different healthcare applications.
- Data quality is important during integration. Patient identifiers must be matched correctly, duplicate records should be removed, missing values should be handled, and different units should be standardized. For example, laboratory systems may use different formats for measurements, so the ETL pipeline must convert them into a common representation before analysis.
- Real-time monitoring can be implemented using Kafka together with Apache Flink or Spark Structured Streaming. When a wearable device detects an abnormal reading, the streaming layer can evaluate the event immediately. An alert can then be sent to healthcare professionals. This is particularly useful for continuous monitoring of patients and early detection of health deterioration.
- Predictive health analytics uses historical patient records and sensor data to develop machine learning models. Features can include age, previous diagnoses, laboratory values, medication history, appointment frequency, wearable measurements, and

previous hospitalizations. Models such as logistic regression, decision trees, random forests, gradient boosting, and neural networks can estimate disease or readmission risk.

- Distributed processing frameworks such as Apache Spark become important when millions of patient records are analyzed. Instead of processing the entire population on one computer, Spark distributes the workload across multiple nodes. This enables large-scale studies involving disease trends, treatment outcomes, regional health patterns, and healthcare resource utilization.
- Machine learning can support diagnosis risk scoring and treatment recommendations. A risk model may calculate the probability that a patient will develop a particular condition or require hospitalization. Recommendation systems can assist clinicians by identifying relevant treatments or similar historical cases. However, such systems should be treated as decision-support tools, with clinical professionals responsible for final decisions.
- Security and governance would include encryption of data both at rest and in transit, role-based access control, strong authentication, audit logging, data masking, and secure backups. Sensitive data used for research can be anonymized or pseudonymized. Every access or modification of patient information should be logged so that unauthorized activity can be detected.
- Compliance mechanisms ensure that the platform follows applicable healthcare and data-protection regulations. Data retention policies, consent management, access reviews, audit trails, and controlled data sharing should be implemented. Disaster recovery and backup systems are also necessary because healthcare records must remain available even after hardware or software failures.
- The complete architecture can therefore be represented as patient records + wearable devices + laboratories + appointments → secure ingestion → ETL/interoperability → hybrid storage → batch/stream processing → machine learning → clinical dashboards and alerts. This architecture enables healthcare organizations to move from simple record keeping toward predictive and data-driven healthcare while maintaining security and privacy.